

Contents

QUIC Migration State Transfer: Comprehensive Comparison	1
1. Research Context	1
2. Test Environment	1
3. Dimension 1: When to Transfer (Timing Strategy)	2
3.1 Immediate Timing	2
3.2 Lazy Timing	2
3.3 Key Difference	2
4. Dimension 2: How to Transfer (Mechanism)	3
4.1 TCP Push	3
4.2 HTTP Pull	3
4.3 Redis KV (Key-Value)	3
4.4 Redis Pub/Sub	3
5. Results: All 8 Combinations	4
6. Evaluation Metrics	5
6.1 Latency	5
6.2 Security	5
6.3 Scalability	5
6.4 Reliability	5
6.5 Coupling	5
6.6 Simplicity	5
6.7 Dependencies	5
6.8 Persistence	5
7. Master Evaluation Table	6
8. Scoring (1-5 scale, 5 is best)	6
9. Recommendations	6

QUIC Migration State Transfer: Comprehensive Comparison

1. Research Context

In cross-machine QUIC server-side migration, the primary server completes the TLS 1.3 handshake with the client and then must transfer the cryptographic state (TLS traffic secrets, Connection IDs, client address) to a preferred server on a different physical machine. The preferred server uses this state to decrypt the client's PATH_CHALLENGE and respond with PATH_RESPONSE, completing the migration transparently.

This document evaluates all possible combinations of **when** and **how** to perform this state transfer, across 8 research metrics, to determine the best approach for different deployment scenarios.

2. Test Environment

Three physical machines on the same LAN (141.217.168.x):

- **Client (141.217.168.127):** Runs neqo-client or Firefox. Completely unmodified. Has no knowledge that two servers are involved in the connection.
- **Primary Server (141.217.168.152, opti7040):** Runs our modified Neqo HTTP/3 server. Completes TLS handshake, advertises preferred_address, exports crypto state (~445 bytes), and transfers it to the preferred server.
- **Preferred Server (141.217.168.143, homeserver2):** Runs our custom preferred-server binary. Receives crypto state, imports TLS keys, listens for QUIC packets, decrypts PATH_CHALLENGE, sends PATH_RESPONSE. Also runs Redis server (port 6379) for Redis-based backends.

All backends and timings are selected via environment variables, allowing rapid switching between configurations without code changes.

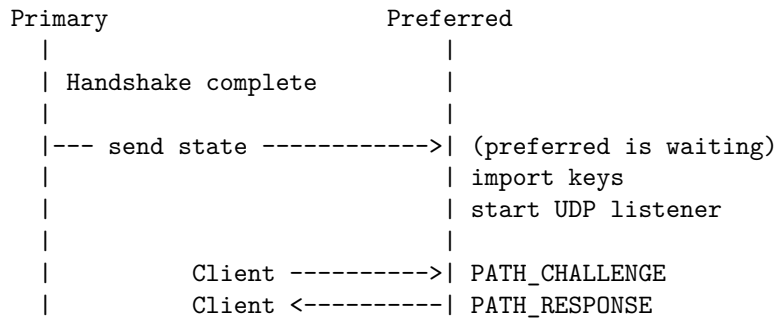
3. Dimension 1: When to Transfer (Timing Strategy)

The timing of state transfer affects latency, security, and which mechanisms are compatible.

3.1 Immediate Timing

The preferred server **blocks and waits** for the migration state before listening for any QUIC packets. Once the state arrives, it imports the crypto keys and then starts the UDP listener.

Flow:

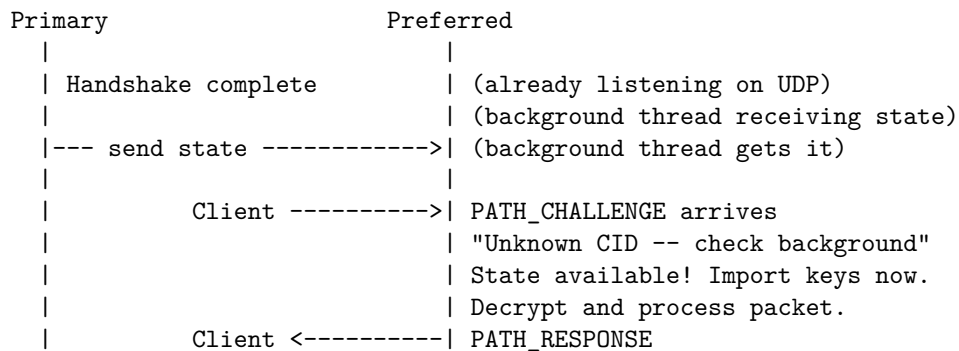


Characteristics: - Simple and predictable - Preferred server is always ready before the first packet arrives - Every connection triggers a state transfer, even if the client never migrates - Works with all 4 transfer mechanisms

3.2 Lazy Timing

The preferred server **starts listening for QUIC packets immediately**. The state receiver runs in a background thread. When the first packet with an unknown Connection ID arrives, the server checks if the state has been received in the background. If yes, it imports the crypto and processes the packet. If not, it waits briefly for the background receiver to deliver.

Flow:



Characteristics: - Crypto import is deferred until actually needed - Preferred server is responsive to packets from the start - Requires state to be available when the first packet arrives (either already received in background, or fetchable on demand) - Slightly more complex implementation (background thread + channel)

3.3 Key Difference

With Immediate timing, the preferred server does nothing until state arrives. With Lazy timing, the preferred server is already listening and the state flows in parallel. Lazy is better for production because it decouples the timing of state arrival from packet arrival.

4. Dimension 2: How to Transfer (Mechanism)

4.1 TCP Push

The primary server opens a direct TCP connection to the preferred server and sends the state as a binary blob: 4-byte length prefix followed by the serialized state bytes. The connection is closed immediately after.

- **Direction:** Primary pushes to preferred (one-way)
- **State persistence:** None. Once sent, it's gone. If nobody is listening, the connection is refused.
- **Scalability:** Primary must know the exact address of the preferred server. Adding servers requires reconfiguration.
- **Security:** State travels in plaintext over TCP. Brief exposure window (send + close).

4.2 HTTP Pull

The primary server starts a lightweight HTTP server after the handshake. The preferred server sends an HTTP GET request to pull the state. The HTTP response contains the state as a binary body.

- **Direction:** Preferred pulls from primary (request-response)
- **State persistence:** In primary server memory until pulled. Survives as long as primary is running.
- **Scalability:** Preferred must know the primary's HTTP address. Better than TCP Push because the preferred server decides when to fetch.
- **Security:** Can be upgraded to HTTPS for encrypted transfer. State stays on the primary until explicitly requested.

4.3 Redis KV (Key-Value)

The primary server writes the state to Redis using SET with a CID-based key and configurable TTL (time-to-live). The preferred server reads it using GET, then deletes the key.

- **Direction:** Decoupled via shared storage (write then read)
- **State persistence:** Yes, until TTL expires or explicitly deleted. Survives server restarts.
- **Scalability:** Excellent. Any primary writes with the CID as key. Any preferred server that receives a packet looks up the CID in Redis. No server needs to know about any other server. Supports true N:M routing.
- **Security:** State is stored in Redis (third-party). Anyone with Redis access can read all migration secrets. Mitigated by Redis AUTH, TLS, and delete-after-read.

4.4 Redis Pub/Sub

The primary server publishes the state to a Redis channel (instance-specific). The preferred server subscribes to the channel and receives the message when it arrives.

- **Direction:** Event-driven push via message broker
- **State persistence:** None. If nobody is subscribed when the message is published, it is lost forever.
- **Scalability:** Good for fan-out (multiple preferred servers can subscribe). But all subscribers receive all messages, which is wasteful and a security concern.
- **Security:** State is broadcast to all subscribers on the channel. Worst security posture of all mechanisms.

5. Results: All 8 Combinations

Every combination of timing strategy and transfer mechanism was tested end-to-end across three physical machines. All achieved 100% success rate.

	TCP Push	HTTP Pull	Redis KV	Redis Pub/Sub
Immediate	PASS send: 734us	PASS send: 35ms	PASS send: 1.2ms	PASS send: 1.5ms
Lazy	PASS send: 479us import: 7us	PASS send: 25ms import: 24ms	PASS send: 1.5ms import: 4ms	PASS send: 1.5ms import: 425us

How Lazy works with TCP Push and Redis Pub/Sub: The preferred server starts the state receiver in a background thread at startup. The TCP listener or Redis subscriber begins waiting immediately, so the state is captured when it arrives. The main thread listens for QUIC packets. When the first packet arrives, the main thread checks if the background thread has received the state. If yes, it imports the keys and processes the packet.

Send latency is measured on the primary server (time from starting the send operation to completion). **Import latency** (lazy only) is measured on the preferred server (time from first packet arrival to crypto import completion).

6. Evaluation Metrics

We evaluate each combination on 8 metrics, each scored on a 1-5 scale (5 is best):

6.1 Latency

How fast the state transfer completes. Lower is better. TCP Push is fastest because it is a direct socket connection with no middleware or protocol overhead. HTTP Pull is slowest due to HTTP headers, TCP handshake, and request-response overhead.

6.2 Security

How well TLS secrets are protected during and after transfer. Considers: encryption in transit, exposure window, at-rest storage, who can intercept, and ease of adding encryption.

- **Best:** Lazy + HTTP Pull – secrets stay in primary memory until explicitly pulled
- **Worst:** Redis Pub/Sub – secrets broadcast to all subscribers

6.3 Scalability

Can the system handle N primary servers and M preferred servers without reconfiguration?

- **Best:** Redis KV – any primary writes with CID key, any preferred reads by CID. Fully decoupled.
- **Worst:** TCP Push – primary must know the exact preferred server address.

6.4 Reliability

What happens when things go wrong? Server crashes, network hiccups, restarts.

- **Best:** Redis KV – state persists in Redis independently of both servers. Survives crashes.
- **Worst:** TCP Push (immediate) – if preferred is down when state is sent, it's lost.

6.5 Coupling

How independent are the servers from each other? Loose coupling is better for maintainability and deployment flexibility.

- **Loose (5):** Redis backends – servers only know the Redis address, not each other
- **Tight (1):** TCP Push – primary must know preferred's exact address and port

6.6 Simplicity

How easy is the code to implement, understand, and maintain?

- **Simplest:** TCP Push – a few lines of TCP connect, write, close
- **More complex:** All others require protocol handling or third-party integration

6.7 Dependencies

Does the mechanism require external infrastructure?

- **No dependencies:** TCP Push, HTTP Pull – pure Rust, no external services
- **Requires Redis:** Redis KV, Redis Pub/Sub – need a running Redis server

6.8 Persistence

Does the state survive if a server restarts between transfer and migration?

- **Persistent:** Redis KV – state in Redis survives until TTL expires
- **Not persistent:** TCP Push, Redis Pub/Sub – state gone after delivery

7. Master Evaluation Table

Combination	Latency	Sec	Scale	Rel	Couple	Sim	Dep	Persist
Imm + TCP Push	734us	LOW	LOW	LOW	TIGHT	HIGH	NONE	NONE
Imm + HTTP Pull	35ms	MED	MED	MED	MED	MED	NONE	MEM
Imm + Redis KV	1.2ms	MED	HIGH	HIGH	LOOSE	MED	REDIS	YES
Imm + Redis PS	1.5ms	LOW	MED+	LOW	LOOSE	MED	REDIS	NONE
Lazy + TCP Push	479us	MED	LOW	MED	TIGHT	MED	NONE	NONE
Lazy + HTTP Pull	25ms	HIGH	MED	MED	MED	MED	NONE	MEM
Lazy + Redis KV	1.5ms	MED	HIGH	HIGH	LOOSE	MED	REDIS	YES
Lazy + Redis PS	1.5ms	LOW	MED+	MED	LOOSE	MED	REDIS	NONE

8. Scoring (1-5 scale, 5 is best)

Combination	Lat	Sec	Sc1	Rel	Cpl	Sim	Dep	Per	TOTAL
Imm + TCP Push	5	1	1	2	1	5	5	1	21
Imm + HTTP Pull	2	3	3	3	3	3	5	2	24
Imm + Redis KV	4	3	5	5	5	3	2	5	32
Imm + Redis PS	4	1	4	2	5	3	2	1	22
Lazy + TCP Push	5	2	1	3	1	3	5	1	21
Lazy + HTTP Pull	2	5	3	3	3	3	5	3	27
Lazy + Redis KV	4	3	5	5	5	3	2	5	32
Lazy + Redis PS	4	1	4	3	5	3	2	1	23

9. Recommendations

Best overall (32/40): Redis KV – works with both Immediate and Lazy timing. Scores highest in scalability (N:M routing by CID), reliability (state survives crashes), persistence (TTL-based expiry), and coupling (fully decoupled). The only downside is requiring a Redis server, which is standard infrastructure in production environments.

Best for security (27/40): Lazy + HTTP Pull – TLS secrets never leave the primary server’s memory until the preferred server explicitly requests them. This minimizes the exposure window. HTTPS can be added to encrypt the transfer channel. No third-party storage means no external system holds your secrets.

Best for demos and testing (21/40): Immediate + TCP Push – the fastest latency (734us) and simplest implementation. No dependencies, no external services. But it scores poorly on security (plaintext), scalability (1:1 only), and coupling (tight). Use this for research demonstrations, not production.

Not recommended: Redis Pub/Sub – it requires the same Redis infrastructure as Redis KV but scores lower in security (secrets are broadcast to all channel subscribers) and persistence (messages are ephemeral – lost if no subscriber is active). There is no practical reason to choose Pub/Sub over KV for this use case.